ЛАБОРАТОРНА РОБОТА № 3
ДОСЛІДЖЕННЯ РОБОТИ З КОМПОНЕНТОМ FRAGMENT

Мета роботи: дослідити способи збереження даних (база даних, файлова система, тощо) та отримати практичні навички щодо використання сховищ даних.

Завдання

Написати програму під платформу Андроїд, яка доповнює програму, що розроблена за лабораторною роботою 2, роботою зі сховищами.
Тобто при натисканні на кнопку «ОК» додатково:

- здійснюється запис результату взаємодії з інтерфейсом до сховища (файл або базу даних);
- користувач інформується відповідним повідомленням щодо успішності запису.

Також інтерфейс необхідно доповнити кнопкою «Відкрити», натискання на яку призводить до переходу на іншу Діяльність, у якій відображається вміст даних, що зберігаються у сховищі. Якщо дані відсутні (сховище пусте) відобразити відповідне повідомлення. За бажанням можна додатково реалізувати оновлення та видалення даних зі сховища.

Варіант №3

Вікно містить групу опцій (радіо-батонів) для вибору шрифту, текстове поле для введення інформації та кнопки «ОК» і «Cancel». Вивести введений рядок обраним шрифтом при натисканні на кнопку «ОК» у деяке текстове поле та очистити при натисканні кнопки «Cancel».

Реалізація

У попередній лабораторній роботі інтерфейс програми складався з двох фрагментів, що працювали в межах однієї Activity. Перший фрагмент містив форму введення даних, де користувач міг ввести текст та обрати один із запропонованих варіантів шрифту. Після натискання кнопки «ОК» введені дані передавалися до другого фрагмента, який відображав результат з відповідним форматуванням. Кнопка «Cancel» дозволяла очистити результат та форму введення.

У даній лабораторній роботі функціональність програми було розширено шляхом реалізації механізму збереження даних у внутрішньому сховищі застосунку. Для цього було використано файлову систему Android, що дозволяє зберігати дані у файлах у внутрішній пам’яті пристрою.

Основною зміною стало додавання механізму збереження введених користувачем даних у файл history.txt. Після введення тексту та вибору шрифту при натисканні кнопки «OK» дані зберігаються у файл у внутрішньому сховищі застосунку. У файл записується рядок, що містить введений текст та назву обраного шрифту. Для запису даних використовується режим MODE_APPEND, що дозволяє додавати нові записи до вже існуючих у файлі без їх перезапису.

Для реалізації цієї функції у першому фрагменті було створено метод збереження даних у файл, який формує рядок для запису та виконує операцію запису за допомогою методу openFileOutput(). У випадку успішного збереження даних користувачу відображається інформаційне повідомлення про успішне виконання операції.

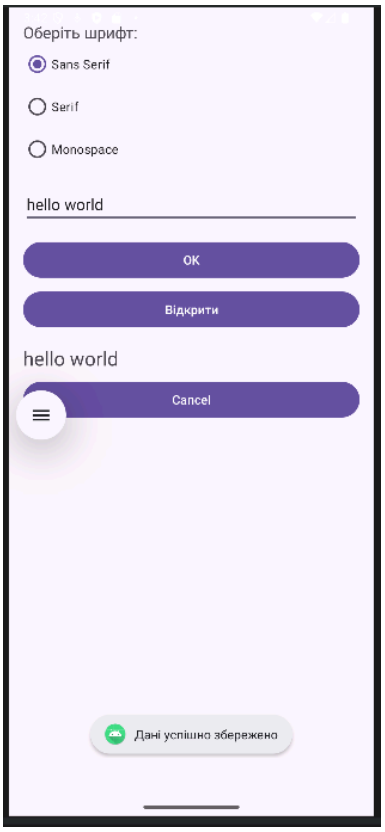

Рисунок 1 – Відображення повідомлення про збереження даних

Крім того, до інтерфейсу першого фрагмента було додано нову кнопку «Відкрити», яка дозволяє перейти до перегляду збережених даних. При натисканні цієї кнопки відкривається новий екран застосунку, реалізований у вигляді окремої Activity.

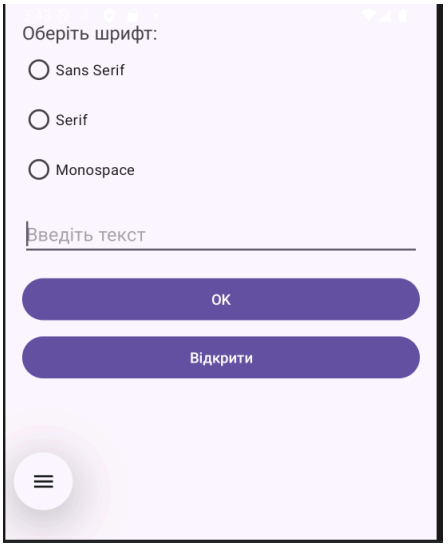

Рисунок 2 – Оновлений інтерфейс програми

Для відображення історії збережених записів було створено нову діяльність HistoryActivity. У цій Activity відбувається зчитування даних із файлу history.txt та їх відображення у текстовому полі. Якщо файл відсутній або не містить записів, користувачу відображається повідомлення про те, що сховище порожнє.

Рисунок 3 – Інформація про порожнє сховище

Інтерфейс екрану історії містить прокручуване текстове поле для відображення збережених записів, кнопку «Очистити історію» для видалення файлу з даними та кнопку «Назад» для повернення до основного екрану застосунку. Кнопка очищення історії виконує видалення файлу history.txt із внутрішнього сховища та оновлює відображення інформації на екрані.

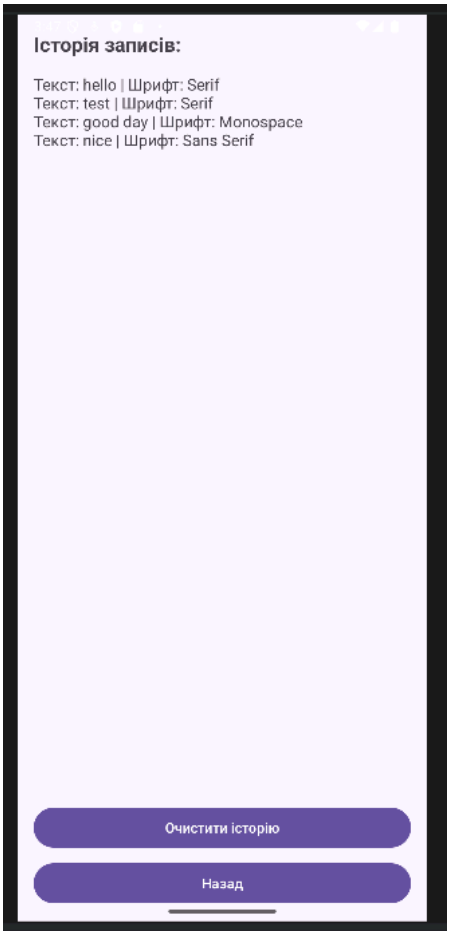

Рисунок 4 – Інтерфейс сховища даних

Таким чином, у програмі було реалізовано повний цикл роботи зі сховищем даних: запис інформації у файл, зчитування збережених даних та можливість їх видалення.

Тестування

У процесі тестування було перевірено коректність роботи механізму збереження та відображення даних у внутрішньому сховищі застосунку.

Спочатку було перевірено роботу програми при введенні коректних даних. Користувач вводить текст у поле введення, обирає один із запропонованих варіантів шрифту та натискає кнопку «OK». Після цього введені дані зберігаються у файл history.txt, а користувачу відображається повідомлення про успішне збереження інформації.

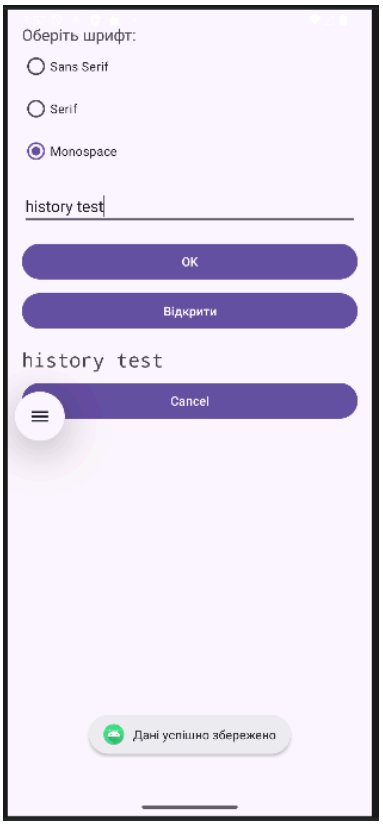

Рисунок 5 – Введення тексту та вибір шрифту та збереження даних

Після натискання кнопки «OK» відбувається запис даних у файл внутрішнього сховища.

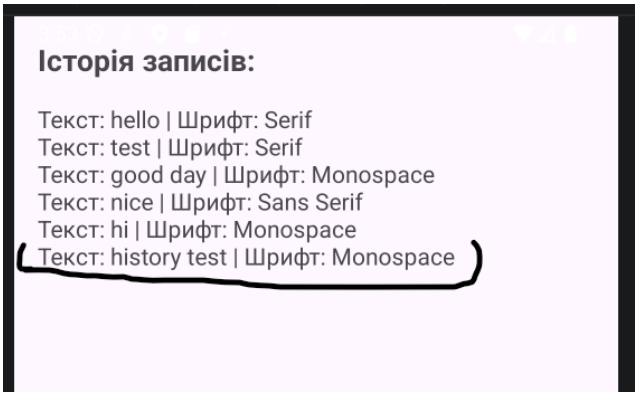

Рисунок 6 – Перевірка запису у файл

Також було перевірено роботу функції очищення сховища. При натисканні кнопки «Очистити історію» файл history.txt видаляється з внутрішнього сховища застосунку, після чого на екрані відображається повідомлення про відсутність записів.

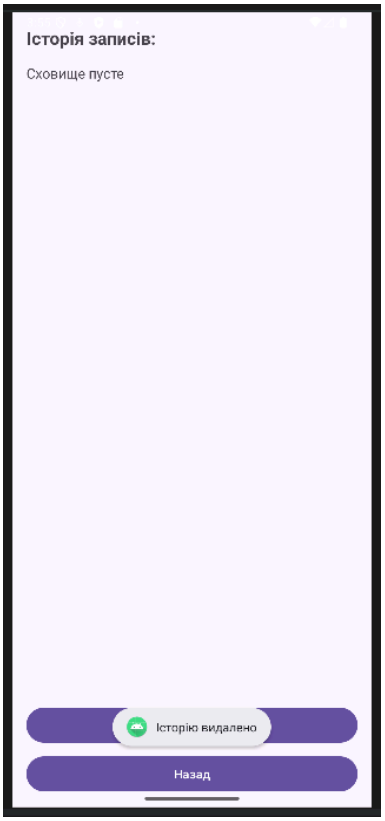

Рисунок 7 – Результат очищення історії збережених даних

У результаті тестування було підтверджено коректну роботу всіх реалізованих функцій: збереження даних у файл, зчитування інформації зі сховища, відображення історії записів та можливість повного очищення збережених даних.

Контрольні питання

1. Опишіть як організована робота з налаштуваннями (даними у вигляді пари «ключ- значення»).

Робота з налаштуваннями у вигляді пари «ключ-значення» в Android зазвичай реалізується за допомогою механізму SharedPreferences. У цьому сховищі дані зберігаються як набір ключів та відповідних їм значень різних типів (String, Int, Boolean тощо). Кожне значення отримується за унікальним ключем, що дозволяє швидко читати або змінювати

налаштування. Такий спосіб використовується для збереження невеликого обсягу даних, наприклад параметрів застосунку або налаштувань користувача.

2. Опишіть типи сховищ файлів та причини їх використання.

У мобільних застосунках використовують внутрішнє та зовнішнє файлове сховище. Внутрішнє сховище доступне лише самому застосунку та використовується для збереження службових або конфіденційних даних. Зовнішнє сховище доступне іншим застосункам та користувачу і застосовується для зберігання великих файлів, таких як медіа або документи. Використання різних типів сховищ дозволяє забезпечити безпеку даних і зручність доступу до них.

3. Опишіть процес роботи з файлами та файловою системою.

Робота з файлами в Android передбачає створення, запис, читання та видалення файлів у файловій системі пристрою. Для запису даних застосовуються потоки введення та виведення, наприклад FileOutputStream та FileInputStream. Файли можуть зберігатися у внутрішній пам’яті або у зовнішньому сховищі. Під час роботи з файлами важливо перевіряти їх існування та коректно закривати потоки після завершення операцій.

4. Опишіть процес роботи з базами даних SQLite за допомогою класу SQLiteOpenHelper, наведіть переваги та недоліки.

Робота з базами даних SQLite часто виконується за допомогою класу SQLiteOpenHelper, який допомагає створювати та оновлювати базу даних. У цьому класі перевизначаються методи onCreate() для створення таблиць та onUpgrade() для оновлення структури бази. Доступ до даних здійснюється через SQL-запити. Перевагою такого підходу є повний контроль над базою даних і висока продуктивність, а недоліком є необхідність вручну писати SQL-запити та більша складність підтримки коду.

5. Як відбувається робота з БД за допомогою бібліотеки Room, наведіть переваги та недоліки.

Бібліотека Room є надбудовою над SQLite, яка спрощує роботу з базами даних у Android. Вона використовує об’єктно-орієнтований підхід, де таблиці представляються у вигляді класів-сутностей, а запити

виконуються через спеціальні інтерфейси DAO. Room автоматично перевіряє коректність запитів під час компіляції та спрощує роботу з даними. Перевагами є зручність використання, менша кількість помилок та зрозумілий код, а недоліками — додатковий рівень абстракції та дещо більша складність налаштування.

6. Наведіть характеристики екранів мобільних пристроїв.

Екрани мобільних пристроїв характеризуються такими параметрами, як розмір діагоналі, роздільна здатність, щільність пікселів (ppi), співвідношення сторін, яскравість та частота оновлення. Важливим показником є щільність пікселів, від якої залежить чіткість зображення. Також значення має тип матриці екрана, що впливає на кольоропередачу, контрастність та енергоспоживання.

7. Наведіть класифікацію та відмінності технологій (типів) екранів мобільних пристроїв.

Основними технологіями екранів мобільних пристроїв є LCD та OLED. LCD-екрани використовують підсвічування та рідкокристалічну матрицю, що забезпечує хорошу яскравість і доступну вартість. OLED-екрани складаються з органічних світлодіодів, які самостійно випромінюють світло, завдяки чому мають високий контраст і глибокий чорний колір. OLED-дисплеї зазвичай споживають менше енергії при відображенні темних кольорів.

8. Наведіть поняття та характеристику сенсорних екранів.

Сенсорний екран — це дисплей, який дозволяє користувачу взаємодіяти з пристроєм за допомогою дотику. Такий екран поєднує функції відображення інформації та введення даних. Сенсорні екрани широко використовуються у смартфонах, планшетах та інших портативних пристроях, оскільки забезпечують інтуїтивне керування та швидку взаємодію з інтерфейсом.

9. Наведіть загальну класифікацію сенсорних екранів.

Сенсорні екрани класифікуються за принципом роботи. Основними типами є резистивні, ємнісні, інфрачервоні та оптичні. Резистивні екрани реагують на фізичний тиск, тоді як ємнісні визначають зміну електричного

поля при дотику пальця. Інфрачервоні та оптичні технології використовують спеціальні датчики для визначення координат дотику.

10. Наведіть рекомендації щодо розробки інтерфейсів для сенсорних екранів.

При розробці інтерфейсів для сенсорних екранів необхідно враховувати зручність взаємодії користувача з елементами керування. Кнопки та інші елементи повинні мати достатній розмір для комфортного натискання. Інтерфейс має бути простим, зрозумілим та адаптованим до різних розмірів екранів. Також важливо уникати перевантаження екрана елементами та забезпечувати швидку реакцію застосунку на дії користувача.

Висновки

У ході виконання лабораторної роботи було досліджено основні способи збереження даних у мобільних застосунках на платформі Android. У процесі роботи було розширено функціональність створеного раніше застосунку та реалізовано механізм збереження введених користувачем даних у внутрішньому файловому сховищі пристрою.

Було реалізовано запис інформації у файл history.txt, що дозволяє зберігати історію введених даних та обраних параметрів шрифту. Також було створено окрему Activity для перегляду збережених записів, у якій реалізовано зчитування інформації з файлу та її відображення на екрані. Додатково було реалізовано можливість очищення історії шляхом видалення файлу зі сховища застосунку.

У результаті виконання лабораторної роботи було набуто практичних навичок роботи з файловою системою Android, зокрема створення, запису, зчитування та видалення файлів у внутрішньому сховищі. Отримані знання дозволяють краще зрозуміти принципи організації збереження даних у мобільних застосунках.

Репозиторій GitHub:
https://github.com/ProfessionalA1k0/lab2_android

Код програми:

MainActivity.kt:

```
package com.example.lab2

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity(),
   InputFragment.Listener,
   ResultFragment.Listener {

   override fun onCreate(savedInstanceState: Bundle?) {
       super.onCreate(savedInstanceState)
       setContentView(R.layout.activity_main)

       if (savedInstanceState == null) {
           supportFragmentManager.beginTransaction()
               .replace(R.id.containerInput, InputFragment())
               .commit()
       }
   }

   override fun onok(text: String, fontId: Int) {
       supportFragmentManager.beginTransaction()
           .replace(R.id.containerResult,
ResultFragment.newInstance(text, fontId))
           .commit()
   }

   override fun onCancel() {


(supportFragmentManager.findFragmentById(R.id.containerInput) as?
InputFragment)
           ?.clearForm()


supportFragmentManager.findFragmentById(R.id.containerResult)?.let
{ result ->
           supportFragmentManager.beginTransaction()
               .remove(result)
               .commit()
       }
   }
}
```

InputFragment.kt:

```
package com.example.lab2

import android.content.Context
import android.content.Intent
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import android.widget.EditText
import android.widget.RadioGroup
import android.widget.Toast
import androidx.fragment.app.Fragment
import java.io.FileOutputStream

class InputFragment : Fragment() {
   interface Listener {
       fun onok(text: String, fontId: Int)
   }

   private var listener: Listener? = null
   private lateinit var rgFont: RadioGroup
   private lateinit var etInput: EditText

   override fun onAttach(context: Context) {
       super.onAttach(context)
       listener = context as? Listener
   }

   override fun onCreateView(
       inflater: LayoutInflater, container: ViewGroup?,
       savedInstanceState: Bundle?
   ): View {
       val view = inflater.inflate(R.layout.fragment_input,
container, false)
       rgFont = view.findViewById(R.id.rgFont)
       etInput = view.findViewById(R.id.etInput)

       val btnOk = view.findViewById<Button>(R.id.btnOk)
       val btnOpen = view.findViewById<Button>(R.id.btnOpenHistory)

       btnOk.setOnClickListener {
           val text = etInput.text.toString()
           val selectedId = rgFont.checkedRadioButtonId

           if (text.isBlank() || selectedId == -1) {
```

```
                Toast.makeText(requireContext(), "Завершіть
введення всіх даних", Toast.LENGTH_SHORT).show()
                return@setOnClickListener
            }

            saveToFile(text, selectedId)

            listener?.onok(text, selectedId)
        }

        btnOpen.setOnClickListener {
            val intent = Intent(requireContext(),
HistoryActivity::class.java)
            startActivity(intent)
        }

        return view
    }

    private fun saveToFile(text: String, fontId: Int) {
        val fontName = when (fontId) {
            R.id.rbSans -> "Sans Serif"
            R.id.rbSerif -> "Serif"
            R.id.rbMono -> "Monospace"
            else -> "Default"
        }

        val dataToSave = "Текст: $text | Шрифт: $fontName\n"

        try {
            val fileOutputStream: FileOutputStream =
                requireContext().openFileOutput("history.txt",
Context.MODE_APPEND)
            fileOutputStream.write(dataToSave.toByteArray())
            fileOutputStream.close()
            Toast.makeText(requireContext(), "Дані успішно
збережено", Toast.LENGTH_SHORT).show()
        } catch (e: Exception) {
            e.printStackTrace()
            Toast.makeText(requireContext(), "Помилка при
збереженні", Toast.LENGTH_SHORT).show()
        }
    }

    fun clearForm() {
        etInput.text.clear()
        rgFont.clearCheck()
    }
```

```
    override fun onDetach() {
        listener = null
        super.onDetach()
    }
}
```

ResultFragment.kt:

```
package com.example.lab2

import android.graphics.Typeface
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import android.widget.TextView
import androidx.fragment.app.Fragment

class ResultFragment : Fragment() {

    interface Listener {
        fun onCancel()
    }

    private var listener: Listener? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        listener = activity as? Listener
    }

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        val view = inflater.inflate(R.layout.fragment_result,
container, false)

        val tvResult = view.findViewById<TextView>(R.id.tvResult)
        val btnCancel = view.findViewById<Button>(R.id.btnCancel)

        val text = requireArguments().getString(ARG_TEXT).orEmpty()
        val fontId = requireArguments().getInt(ARG_FONT_ID)

        val typeface = when (fontId) {
            R.id.rbSans -> Typeface.SANS_SERIF
            R.id.rbSerif -> Typeface.SERIF
```

```
            R.id.rbMono -> Typeface.MONOSPACE
            else -> Typeface.DEFAULT
        }

        tvResult.text = text
        tvResult.typeface = typeface

        btnCancel.setOnClickListener {
            listener?.onCancel()
        }

        return view
    }

    companion object {
        private const val ARG_TEXT = "text"
        private const val ARG_FONT_ID = "fontId"

        fun newInstance(text: String, fontId: Int): ResultFragment
{
            return ResultFragment().apply {
                arguments = Bundle().apply {
                    putString(ARG_TEXT, text)
                    putInt(ARG_FONT_ID, fontId)
                }
            }
        }
    }
}
```

HistoryActivity.kt:

```
package com.example.lab2

import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import java.io.File

class HistoryActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_history)

        val tvContent =
findViewById<TextView>(R.id.tvHistoryContent)
```

```
        val btnBack = findViewById<Button>(R.id.btnBack)
        val btnDelete = findViewById<Button>(R.id.btnDeleteHistory)

        displayHistory(tvContent)

        btnBack.setOnClickListener {
            finish()
        }

        btnDelete.setOnClickListener {
            val file = File(filesDir, "history.txt")
            if (file.exists()) {
                file.delete()
                tvContent.text = "Сховище пусте"
                Toast.makeText(this, "Історію видалено",
Toast.LENGTH_SHORT).show()
            }
        }
    }

    private fun displayHistory(textView: TextView) {
        val file = File(filesDir, "history.txt")
        if (file.exists() && file.length() > 0L) {
            val content = file.readText()
            textView.text = content
        } else {
            textView.text = "Сховище пусте"
        }
    }
}
```

activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <FrameLayout
        android:id="@+id/containerInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <FrameLayout
        android:id="@+id/containerResult"
        android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp" />
</LinearLayout>
```

fragment_input.xml:

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"

    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Оберіть шрифт:"
        android:textSize="18sp"/>

    <RadioGroup
        android:id="@+id/rgFont"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <RadioButton
            android:id="@+id/rbSans"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Sans Serif"/>

        <RadioButton
            android:id="@+id/rbSerif"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Serif"/>

        <RadioButton
            android:id="@+id/rbMono"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Monospace"/>

    </RadioGroup>

    <EditText
        android:id="@+id/etInput"
        android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
        android:hint="Введіть текст"
        android:layout_marginTop="16dp"/>

    <Button
        android:id="@+id/btnOk"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="OK"
        android:layout_marginTop="16dp"/>

    <Button
        android:id="@+id/btnOpenHistory"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Відкрити"
        android:layout_marginTop="8dp" />

</LinearLayout>
```

fragment_result.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">

    <TextView
        android:id="@+id/tvResult"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="22sp" />

    <Button
        android:id="@+id/btnCancel"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Cancel"
        android:layout_marginTop="8dp" />
</LinearLayout>
```

activity_history.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:id="@+id/tvHistoryTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Історія записів:"
        android:textSize="20sp"
        android:textStyle="bold"
        android:layout_marginBottom="16dp" />

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1">
        <TextView
            android:id="@+id/tvHistoryContent"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textSize="16sp" />
    </ScrollView>

    <Button
        android:id="@+id/btnDeleteHistory"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Очистити історію"
        android:layout_marginTop="8dp" />

    <Button
        android:id="@+id/btnBack"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Назад"
        android:layout_marginTop="8dp" />
</LinearLayout>
```